

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №7

Робота з Redis

Виконала
Студентка 3 курсу
Групи МІТ-31
Факультету інформаційних технологій
Кубедінова Поліна

Київ, 2026

Тема: Робота з Redis

Мета роботи: Закріпити розуміння роботи Redis та навчитися використовувати його основні можливості.

Хід роботи

1. Встановлення Redis та запуск

```
Administrator: Командний рядок - wsl --install -d Ubuntu
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>wsl --install -d Ubuntu
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: polin
New password:
Retype new password:
passwd: password updated successfully
ssermod: no changes
elp improve Ubuntu!

elp us improve Ubuntu features and compatibility by sharing system reports with Canonical.
eports are sent anonymously and do not contain any personal data.
or legal details, please visit: https://ubuntu.com/legal/systems-information-notice

e will save your answer to Windows and will only ask you once.

ould you like to opt-in to platform metrics collection (Y/n)? To see an example of the data collected, enter 'e'.
Y/n/e]: y

Get:31 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 c-n-f Metadata [8276 B]
Get:32 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [78.1 kB]
Get:33 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [24.2 kB]
Get:34 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [2844 B]
Get:35 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 c-n-f Metadata [1132 B]
Get:36 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [40.7 kB]
Get:37 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [13.3 kB]
Get:38 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [20.6 kB]
Get:39 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 c-n-f Metadata [624 B]
Get:40 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [52.3 kB]
Get:41 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [11.1 kB]
Get:42 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Components [216 B]
Get:43 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 c-n-f Metadata [116 B]
Get:44 http://archive.ubuntu.com/ubuntu resolute-backports/main amd64 Components [212 B]
Get:45 http://archive.ubuntu.com/ubuntu resolute-backports/main amd64 c-n-f Metadata [112 B]
Get:46 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [216 B]
Get:47 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 c-n-f Metadata [116 B]
Get:48 http://archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 Components [216 B]
Get:49 http://archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 c-n-f Metadata [120 B]
Get:50 http://archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 Components [216 B]
Get:51 http://archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Fetched 31.2 MB in 4s (8076 kB/s)
23 packages can be upgraded. Run 'apt list --upgradable' to see them.
polin@Polin:/mnt/c/Windows/System32$
```

```

Download size: 1424 kB
Space needed: 7477 kB / 1025 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 liblzfl amd64 3.6-4build1 [7490 B]
Get:2 http://archive.ubuntu.com/ubuntu resolute/universe amd64 redis-tools amd64 5:8.0.5-1 [1363 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute/universe amd64 redis-server amd64 5:8.0.5-1 [53.2 kB]
Fetched 1424 kB in 0s (4635 kB/s)
Selecting previously unselected package liblzfl:amd64.
(Reading database ... 35923 files and directories currently installed.)
Preparing to unpack .../liblzfl_3.6-4build1_amd64.deb ...
Unpacking liblzfl:amd64 (3.6-4build1) ...
Selecting previously unselected package redis-tools.
Preparing to unpack .../redis-tools_5%3a8.0.5-1_amd64.deb ...
Unpacking redis-tools (5:8.0.5-1) ...
Selecting previously unselected package redis-server.
Preparing to unpack .../redis-server_5%3a8.0.5-1_amd64.deb ...
Unpacking redis-server (5:8.0.5-1) ...
Setting up liblzfl:amd64 (3.6-4build1) ...
Setting up redis-tools (5:8.0.5-1) ...
Setting up redis-server (5:8.0.5-1) ...
Created symlink '/etc/systemd/system/redis.service' → '/usr/lib/systemd/system/redis-server.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/redis-server.service' → '/usr/lib/systemd/system/redis-server.service'.
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
polin@Polin:/mnt/c/Windows/System32$

```

```

Processing triggers for libc-bin (2.43-2ubuntu2) ...
polin@Polin:/mnt/c/Windows/System32$ sudo service redis-server start
polin@Polin:/mnt/c/Windows/System32$ sudo service redis-server status
● redis-server.service - Advanced key-value store
   Loaded: loaded (/usr/lib/systemd/system/redis-server.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-16 14:50:27 CEST; 53s ago
 Invocation: 8d1fbf3635d147e583eea3fd8eec3552
    Docs: http://redis.io/documentation,
          man:redis-server(1)
   Main PID: 1176 (redis-server)
   Status: "Ready to accept connections"
     Tasks: 6 (limit: 18872)
    Memory: 4.4M (peak: 5.9M)
       CPU: 165ms
    CGroup: /system.slice/redis-server.service
            └─1176 "/usr/bin/redis-server 127.0.0.1:6379"

May 16 14:50:27 Polin systemd[1]: Starting redis-server.service - Advanced key-value store...
May 16 14:50:27 Polin systemd[1]: Started redis-server.service - Advanced key-value store.
polin@Polin:/mnt/c/Windows/System32$

```

Перевірка підключення через redis-cli

```

May 16 14:50:27 Polin systemd[1]: Started redis-server.service - Advanced key-value store.
polin@Polin:/mnt/c/Windows/System32$ redis-cli
127.0.0.1:6379>

```

Тест PING

```

polin@Polin:/mnt/c/Windows/System32$ redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379>

```

2. Операції з рядками

Створення ключа зі своїм ім'ям

```

127.0.0.1:6379> SET student "Polina"
OK
127.0.0.1:6379>

```

Отримайте значення цього ключа

```
127.0.0.1:6379> GET student
"Polina"
127.0.0.1:6379>
```

Робота з лічильником (INCR)

```
127.0.0.1:6379> INCR mycounter
(integer) 1
127.0.0.1:6379> INCR mycounter
(integer) 2
127.0.0.1:6379>
```

Перевірка фінального значення

```
127.0.0.1:6379> GET mycounter
"2"
127.0.0.1:6379>
```

3. Структури даних

Список (List)

```
127.0.0.1:6379> LPUSH tasks "Task1"
(integer) 1
127.0.0.1:6379> LPUSH tasks "Task2"
(integer) 2
127.0.0.1:6379> LRANGE tasks 0 -1
1) "Task2"
2) "Task1"
127.0.0.1:6379> LPOP tasks
"Task2"
127.0.0.1:6379> LRANGE tasks 0 -1
1) "Task1"
127.0.0.1:6379>
```

Множина (Set)

```
127.0.0.1:6379> SADD tech:set "Redis"
(integer) 1
127.0.0.1:6379> SADD tech:set "PostgreSQL"
(integer) 1
127.0.0.1:6379> SADD tech:set "MongoDB"
(integer) 1
127.0.0.1:6379> SMEMBERS tech:set
1) "Redis"
2) "PostgreSQL"
3) "MongoDB"
127.0.0.1:6379> SISMEMBER tech:set "Redis"
(integer) 1
127.0.0.1:6379> SMEMBERS tech:set
```

Хеш (Hash)

```

127.0.0.1:6379> HSET profile name "Polina"
(integer) 1
127.0.0.1:6379> HSET profile city "Kyiv"
(integer) 1
127.0.0.1:6379> HGET profile name
"Polina"
127.0.0.1:6379> HGETALL profile
1) "name"
2) "Polina"
3) "city"
4) "Kyiv"
127.0.0.1:6379>

```

Sorted Set

```

127.0.0.1:6379> ZADD scores 85 "Student1"
(integer) 1
127.0.0.1:6379> ZADD scores 92 "Student2"
(integer) 1
127.0.0.1:6379> ZADD scores 74 "Student3"
(integer) 1
127.0.0.1:6379> ZREVRANGE scores 0 1 WITHSCORES
1) "Student2"
2) "92"
3) "Student1"
4) "85"
127.0.0.1:6379> ZRANK scores "Student1"
(integer) 1
127.0.0.1:6379> ZREVRANGE scores 0 -1 WITHSCORES
1) "Student2"
2) "92"
3) "Student1"
4) "85"
5) "Student3"
6) "74"
127.0.0.1:6379>

```

4. Працюємо з TTL

Створюємо тимчасовий ключ `temp:data` зі значенням `"Hello"`, який зникне через **10 секунд (EX 10)**:

```

127.0.0.1:6379> SET temp:data "Hello" EX 10
OK

```

Одразу ж перевіряємо, чи існує ключ та скільки часу йому залишилося жити

```

127.0.0.1:6379> GET temp:data
temp:d"Hello"
127.0.0.1:6379> TTL temp:data
(integer) 10

```

Зачекаймо 10 секунд і спробуймо знову виконати ті ж команди

```
127.0.0.1:6379> GET temp:data
temp:data(nil)
127.0.0.1:6379> TTL temp:data
(integer) -2
```

5. Міні-програма з Redis (додатково)

```
import redis

def main():
    print("=== Запуск міні-програми для роботи з Redis ===")

    try:
        r = redis.Redis(host='localhost', port=6379,
decode_responses=True)

        if r.ping():
            print("[OK] Успішно підключено до локального сервера Redis!")
        except redis.ConnectionError:
            print("[ПОМИЛКА] Не вдалося підключитися до Redis.")
            print("Будь ласка, переконайтеся, що служба запущена у WSL командою: sudo service redis-server start")
            return

        counter_value = r.incr('mycounter')
        print(f"\n[Крок 1] Лічильник 'mycounter' успішно збільшено!")
        print(f" -> Поточне значення лічильника: {counter_value}")

        print("\n[Крок 2] Зчитування списку задач 'tasks'...")
        tasks = r.lrange('tasks', 0, -1)

        if tasks:
            print(" -> Знайдені задачі у списку:")
            for index, task in enumerate(tasks, start=1):
                print(f"    {index}. {task}")
        else:
            print(" -> [Увага] Список задач порожній. Ви можете додати їх у redis-cli за допомогою команди: LPUSH tasks 'Task1'")

        channel_name = 'news'
        message_text = "Привіт! Це повідомлення відправлене автоматично з нашої програми на Python!"

        print(f"\n[Крок 3] Публікація повідомлення в канал '{channel_name}'...")
        subscribers_count = r.publish(channel_name, message_text)
```

```

    print(f" -> Текст повідомлення: \"{message_text}\"")
    print(f" -> Кількість активних підписників, які отримали
повідомлення: {subscribers_count}")
    print("\n=== Роботу програми успішно завершено ===")

if __name__ == "__main__":
    main()

```

КОД

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  GITLENS  DEBUG CONSOLE

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

D:\polin\University>python -u "d:\polin\University\Databases_Kubiedinova\tempCodeRunnerFile.py"
=== Запуск міні-програми для роботи з Redis ===
[OK] Успішно підключено до локального сервера Redis!

[Крок 1] Лічильник 'myscounter' успішно збільшено!
-> Поточне значення лічильника: 3

[Крок 2] Зчитування списку задач 'tasks'...
-> Знайдені задачі у списку:
    1. Task1

[Крок 3] Публікація повідомлення в канал 'news'...
-> Текст повідомлення: "Привіт! Це повідомлення відправлене автоматично з нашої програми на Python!"
-> Кількість активних підписників, які отримали повідомлення: 0

=== Роботу програми успішно завершено ===

```

Результат виконання

6. Аналіз даних (теоретичне завдання)

У реальних проєктах Redis найчастіше використовується як надшвидкий кеш (Caching) для тимчасового збереження результатів важких запитів до реляційних баз даних, що значно прискорює роботу вебсайтів. Також він є незамінним інструментом для керування сесіями користувачів (Session Store) завдяки миттєвому доступу до авторизаційних токенів у оперативній пам'яті. Завдяки підтримці TTL та швидких атомарних лічильників (таких як метод **incr**, який ви виділили у файлі Canvas), Redis часто застосовують для захисту API від перевантажень шляхом обмеження частоти запитів (Rate Limiting). Крім того, вбудовані структури даних і механізми Pub/Sub дозволяють використовувати його як легкий брокер повідомлень для мікросервісів або систем чатів у реальному часі. Нарешті, завдяки відсортованим множинам Redis ідеально підходить для миттєвого прорахунку складних таблиць лідерів та рейтингів в онлайн-іграх.

Висновок

У ході виконання лабораторної роботи №7 було успішно освоєно процес встановлення, налаштування та запуску швидкої NoSQL СУБД Redis у середовищі Windows за допомогою підсистеми WSL

(Ubuntu). На практиці було детально вивчено базові консольні команди для взаємодії з сервером, а також закріплено навички роботи з ключами типу "рядок" та керування часом їхнього життя за допомогою механізму TTL. Окрім простих рядків, ми дослідили складніші вбудовані структури даних Redis, такі як списки, невідсортовані та відсортовані множини, а також хеші, які дозволяють зручно структурувати інформацію в оперативній пам'яті. Додатково було розроблено та протестовано автоматизований скрипт на мові програмування Python, що продемонстрував можливість легкого програмного підключення до бази даних, зчитування інформації та роботи з лічильниками. Отриманий досвід довів надзвичайну ефективність та швидкодію In-Memory рішень, що є критично важливим при розробці високонавантажених систем, побудові систем кешування та швидких сховищ даних у реальних проєктах.